

FIN-220 Hands-On Machine Learning

HW 5

Due on November 17, 2025

1. What is the fundamental idea behind support vector machines?

A support vector machine is a machine learning model that works for linear and nonlinear classification, regression, and novelty detection.

The fundamental idea is illustrated in using SVM for classification. Find a model (fit a line) that best separates two classes (observed data on measured features) while staying as far away from the closest training instance as possible.

Visualize an SMV as fitting the widest possible street between the observed classes

2. What is a support vector?

Support Vectors are the observed class instances that define the decision boundary (the shoulders of the street). Each data instance is represented as a vector of features – thus support vectors. If you remove instances of observed data away from the decision boundary the decision boundary would remain the same.

3. Why is it important to scale the inputs when using SVMs?

SVMs are sensitive to feature scales, and work best when features are put through Scikit-Learn's StandardScaler class's fit_transform() method on the data

4. Can an SVM classifier output a confidence score when it classifies an instance? What about a probability?

Using; from sklearn.svm import LinearSVC,

Configuring the svm with the LinearSVC -

```
svm_clf = make_pipeline(StandardScaler(),
```

```
    LinearSVC(C=1, dual=True, random_state=42))
```

and fitting a model - svm_clf.fit(X, y)

the calling: svm_clf.predict(X_new) on some vector (X_new) of observation will return an array of Boolean value indicating membership in the fitted y class.

You can then call: `svm_clf.decision_function(X_new)` which returns a signed distance from the model fit. Note: This is not scaled in standard deviation units – not directly interpretable as probability “confidence interval” but give you a confidence score if you understand the data.

Support Vector Machines do not naturally output probabilities. For probability estimates you need additional calculations with a cross-validation procedure.

```
svm_clf = SVC(probability=True)
```

```
svm_clf.fit(X_train, y_train)
```

Now you can get probability estimates

```
svm_clf.predict_proba(X_test) # Returns probability for each class
```

5. Say you’ve trained an SVM classifier with an RBF kernel, but it seems to underfit the training set. Should you increase or decrease γ ? What about C ?

The Radial Basis Function kernel aka the Gaussian kernel measures the similarity between two data points based on their Euclidean distance -

square root($(x_2 - x_1)^2 + (y_2 - y_1)^2$)

or in higher dimensions:

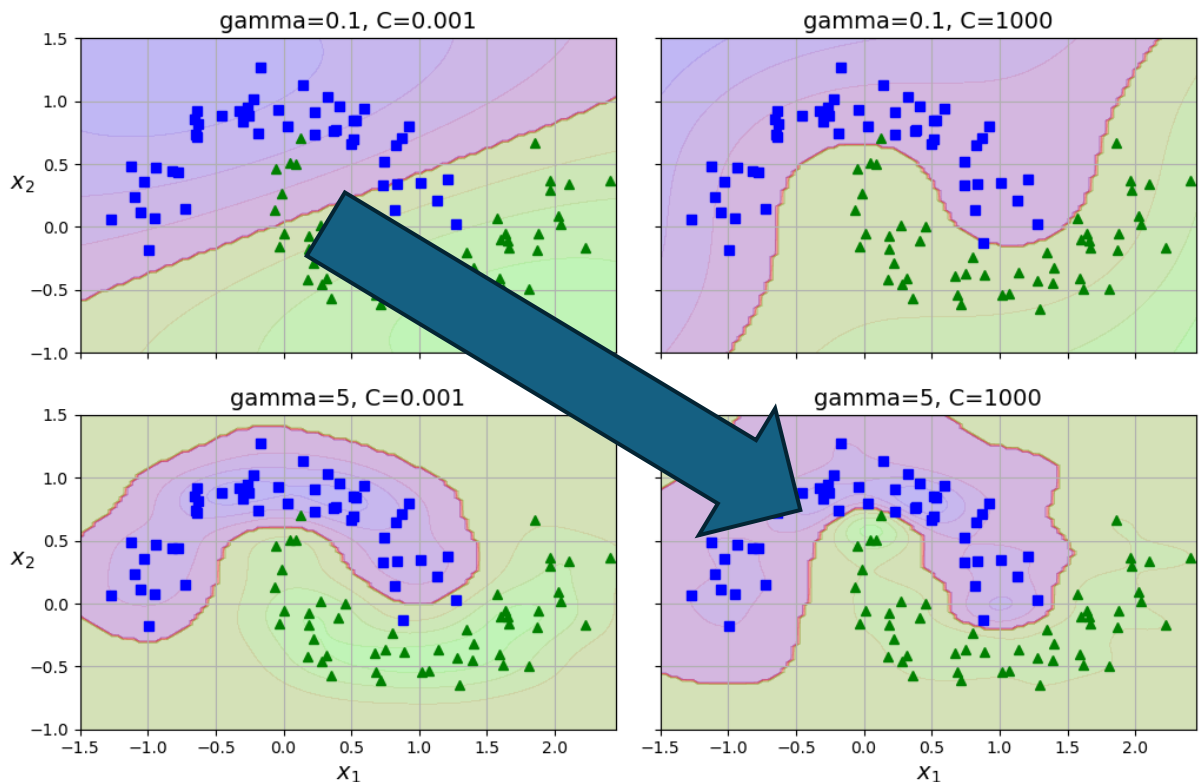
$$d = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Underfitting means the model is too simple, the bias is high – there is more information in the data.

You should increase γ . It will make the boundary more wiggly, more complex, each training observation should have narrower influence on the boundary.

You should increase C . Higher C will create narrower margins, penalizing errors more heavily.

Move from the upper left to the lower right



6. What does it mean for a model to be ϵ -insensitive?

We are talking about SMV regression – Scikit-Learn’s LinearSVR

```
svm_reg = make_pipeline(StandardScaler(),  
                        LinearSVR(epsilon=0.5, dual=True, random_state=42))
```

LinearSVR ignores errors smaller than epsilon.

SVM regression tries to fit as many instance as possible within the “street”. If you add more training data on the street it will not affect the model’s predictions. Thus the model is said to be ϵ -insensitive

7. What is the point of using the kernel trick?

Many datasets are not linearly separable in the original space. You can manually add polynomial features and train a linear SVM on the new space, but it is computationally expensive. Adding polynomial features explodes the number of features.

The kernel trick achieves the same results without computing the transformation.

